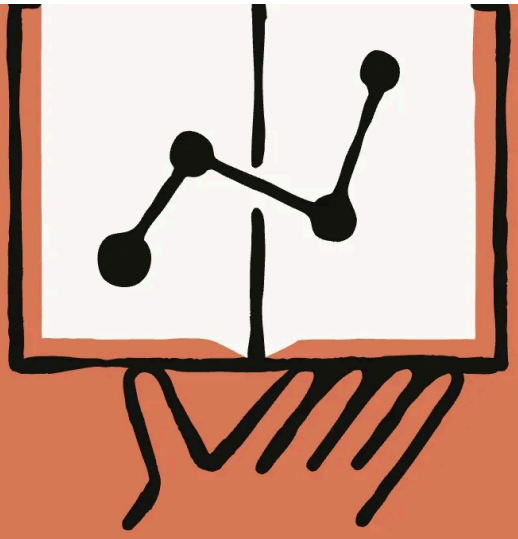

Claude Cowork Plugins: The Complete Implementation Guide

Table of Contents

1. **What Are Claude Cowork Plugins?** — Overview & Key Capabilities
2. **System Requirements & Setup** — Installation Process
3. **Plugin Architecture Deep Dive** — The 5 Components
4. **Step-by-Step Setup Guide** — Installing & Customizing Plugins
5. **Fully Possible Use Cases** — 10 Real-World Workflows
6. **Creating Custom Plugins** — Build Your Own
7. **Real-World Plugin Examples** — Sales, Finance, Product

✅ What Are Claude Cowork Plugins?

Overview

Claude Cowork plugins are modular extensions that transform Claude from a conversational AI into a specialized automation platform tailored to specific roles, teams, and workflows. Launched on **January 30, 2026**, the plugin system enables users to bundle multiple customization types into single installable packages.

Plugins bring the same agentic architecture from Claude Code (the developer-focused tool) to Claude Cowork (the knowledge work productivity tool), without requiring terminal access or coding expertise.

Core Concept

Instead of configuring Claude from scratch for each task, plugins provide ready-made bundles containing:

- **Slash Commands**: Quick shortcuts for common workflows (e.g., `/rfp:extract`, `/legal:discover`)
- **Sub-agents**: Specialized AI workers with isolated contexts for parallel task execution
- **MCP Servers**: Connections to external tools and data sources (Slack, Salesforce, Google Workspace)
- **Hooks**: Automation triggers at lifecycle events (e.g., auto-format after file edits)
- **Skills**: Instructions Claude reads automatically based on task context

Real-World Analogy: Think of plugins as "app stores" for Claude—instead of using a generic assistant, you install plugins that turn Claude into a specialized RFP analyst, financial auditor, or legal document reviewer.

Key Capabilities

Capability	Description	Example
Direct File Access	Read/write local files without manual uploads	Process 200 resumes in a folder → ranked Excel spreadsheet
Sub-agent Coordination	Break complex work into parallel workstreams	5-year financial analysis: one sub-agent per fiscal year
Professional Outputs	Generate formatted deliverables	Excel with formulas, PowerPoint decks, formatted Word docs
Long-Running Tasks	Extended execution without timeouts	Analyze 50 contracts for risk clauses (30+ minute task)
Browser Integration	Control web browsers via Chrome extension	Scrape competitor pricing from websites

Evidence: NASA used custom Claude plugins to generate Mars rover driving instructions, achieving **50% time savings** on a previously manual process.

✅ System Requirements & Setup

Requirements

Requirement	Specification	Notes
Operating System	macOS only	Windows support coming (timeline TBD)
Application	Claude Desktop app	Not available on web or mobile
Subscription	Paid plan required	Pro ($20/month), Max, Team, or Enterprise
Internet	Active connection	Required throughout session
Permissions	Folder access grants	You control which folders Claude can access

Installation Process

Step 1: Install Claude Desktop

```
> Loading JavaScript code...
```

Step 2: Access Cowork

```
> Loading JavaScript code...
```

Step 3: Grant File Access (First Time)

```
> Loading JavaScript code...
```

Security Note: Claude can only access files in folders you explicitly grant. Nothing is accessible by default.

Step 4: Verify Setup

Test with a simple task:

```
> Loading JavaScript code...
```

If Claude responds with a plan and begins working, you're set up correctly.

✅ Plugin Architecture Deep Dive

The Five Plugin Components

Plugins bundle up to five distinct extension types, each serving a specific automation purpose:

```
> Loading JavaScript code...
```

Component #1 — Slash Commands

Purpose: Reusable text-based shortcuts for common workflows

File Format: Markdown files in `commands/` directory

Invocation: Manual (`/plugin-name:command-name`)

Example: `/rfp:extract`

```
> Loading JavaScript code...
```

How It Works:

- User types `/rfp:extract /path/to/rfp-folder/`
- `$ARGUMENTS` placeholder is replaced with `/path/to/rfp-folder/`
- Claude executes the workflow described in the command

Component #2 — Sub-agents

Purpose: Specialized AI workers with isolated contexts for parallel execution

Architecture: Each sub-agent has separate context window, custom system prompt, scoped tool permissions

File Format: Markdown with YAML frontmatter in `agents/` directory

Example: Requirement Extractor Sub-agent

```
> Loading JavaScript code...
```

Key Innovation: Sub-agents report only summary findings back to main orchestrator, preventing context pollution when processing large documents.

Component #3 — MCP Servers (Model Context Protocol)

Purpose: Connect Claude to external tools and data sources

File Format: `.mcp.json` at plugin root

Example: CRM Integration

```
> Loading JavaScript code...
```

Available Integrations (via MCP ecosystem)

- **Productivity**: Slack, Asana, Linear, Jira, Notion
- **Cloud Storage**: Google Drive, Dropbox, Box
- **CRM**: Salesforce, HubSpot
- **Development**: GitHub, GitLab, Sentry
- **Data**: PostgreSQL, MongoDB, Redis
- **Analytics**: Amplitude, Mixpanel, Google Analytics

How It Works:

- MCP servers expose tools (functions), resources (data), and prompts (templates) through standardized JSON-RPC interface
- Claude invokes these tools seamlessly alongside built-in capabilities
- Servers start automatically when plugin is enabled

Component #4 — Hooks

Purpose: Lifecycle event automation (validation, enrichment, blocking)

File Format: `hooks/hooks.json`

Available Events

- `PreToolUse`: Before Claude uses any tool
- `PostToolUse`: After successful tool execution
- `SessionStart`: At beginning of session
- `UserPromptSubmit`: When user submits prompt
- `SubagentStart`: When sub-agent launches

Example: Auto-Format Hook

```
> Loading JavaScript code...
```

Hook Decision Flow

Hooks execute shell scripts that return JSON:

```
> Loading JavaScript code...
```

Component #5 — Skills

Purpose: Instructions Claude reads automatically when relevant to task

File Format: `SKILL.md` files in subdirectories of `skills/`

Example: Legal Clause Taxonomy Skill

```
> Loading JavaScript code...
```

Difference from Commands

- **Commands**: User manually invokes (`/legal:discover`)
- **Skills**: Claude automatically uses when task context matches (e.g., user uploads contract → Legal Taxonomy skill activates)

Plugin Manifest Schema

The `plugin.json` file defines plugin metadata. Complete schema:

```
> Loading JavaScript code...
```

Critical Rules

- ✅ Manifest location: `.claude-plugin/plugin.json` (MUST be in this directory)

- ✅ Component directories: `commands/`, `agents/`, `skills/`, `hooks/` MUST be at plugin root (NOT inside `.claude-plugin/`)
- ✅ Path references: Use `${CLAUDE_PLUGIN_ROOT}` variable for portability
- ✅ Naming convention: kebab-case for all files and directories

---

✅ Step-by-Step Setup Guide

Installing Pre-Built Plugins

Anthropic provides 10+ official plugins for common functions:

Official Plugin Library:

- **Productivity**: Task management, calendar, workflows
- **Enterprise Search**: Find info across company tools
- **Sales**: Prospect research, deal prep
- **Finance**: Financial analysis, modeling, metrics
- **Data**: Query, visualize, interpret datasets
- **Legal**: Document review, risk flagging, compliance
- **Marketing**: Content drafting, campaign planning
- **Customer Support**: Triage issues, draft responses
- **Product Management**: Specs, roadmaps, prioritization
- **Biology Research**: Literature search, result analysis

Installation Process

```
> Loading JavaScript code...
```

Alternative: Upload Custom Plugin

```
> Loading JavaScript code...
```

Using Installed Plugins

- Type `/` or click `+` button to see available commands
- Commands appear as `/plugin-name:command-name`
- Example: If you installed "sales" plugin, you'll see `/sales:research-prospect`, `/sales:prep-discovery`, etc.

Customizing Plugins

After installing, tailor plugins to your workflow:

```
> Loading JavaScript code...
```

Example Customization

Default Sales Plugin:

- Uses generic prospect research (LinkedIn, Crunchbase)
- Outputs to Excel

After Customization:

- Connects to your Salesforce instance (via MCP)
- Pulls existing customer data to identify upsell opportunities
- Outputs to Google Sheets shared with sales team
- Includes your company's sales methodology (MEDDIC, SPIN, etc.) in Skills

---

Creating a Simple Plugin from Scratch

Use Case: Create a "Resume Screener" plugin for HR workflows

Step 1: Create Plugin Directory

```
> Loading JavaScript code...
```

Step 2: Create Manifest

Create `.claude-plugin/plugin.json`:

```
> Loading JavaScript code...
```

Step 3: Create Slash Command

Create `commands/screen-resumes.md`:

```
> Loading JavaScript code...
```

Step 4: Create Sub-agent for Resume Parsing

Create `agents/resume-parser.md`:

```
> Loading JavaScript code...
```

Step 5: Create Skills (Optional)

Create `skills/ats-criteria/SKILL.md`:

```
> Loading JavaScript code...
```

Step 6: Test Locally

```
> Loading JavaScript code...
```

Step 7: Install Plugin

Once tested, install permanently:

Option A: Install to User Scope (available across all projects)

```
> Loading JavaScript code...
```

Option B: Install to Project Scope (shared with team via git)

```
> Loading JavaScript code...
```

Option C: Upload via Cowork UI

```
> Loading JavaScript code...
```

Plugin Name: rfp-assassin

Components:

- Slash Command: /rfp:extract <folder-path>
- Sub-agents:
  - requirement-extractor.md (parallel per section)
  - criteria-parser.md (evaluation rubrics)
  - timeline-mapper.md (milestones, deadlines)
- Skills:
  - rfp-taxonomy/ (SOW structures, compliance matrices)
  - requirement-classification/ (mandatory vs optional logic)
- Hook: PostToolUse → auto-format Excel with conditional formatting
- MCP: Salesforce (pull existing opportunity data)

```
> Loading JavaScript code...
```

/rfp:extract /RFPs/Acme-Corp-RFP-2026/

```
> Loading JavaScript code...
```

```
// From requirement-extractor-1
{
    "section": "Technical Requirements",
    "requirements": [
        {
            "id": "REQ-001",
            "category": "Integration",
            "text": "System must integrate with SAP ERP via REST API",
            "mandatory": true,
            "page": 23,
            "keywords": ["SAP", "REST API", "integration"]
        },
        {
            "id": "REQ-002",
            "category": "Security",
            "text": "All data must be encrypted at rest using AES-256",
            "mandatory": true,
            "page": 24,
            "keywords": ["encryption", "AES-256", "data at rest"]
        }
        // ... 85 more requirements
    ]
}
```

```
> Loading JavaScript code...
```

Plugin Name: 10k-analyst

Components:

- Slash Command: /finance:analyze-10k <ticker> <years>
- Sub-agents (one per fiscal year):
  - 10k-extractor-FY2021.md
  - 10k-extractor-FY2022.md
  - 10k-extractor-FY2023.md
  - 10k-extractor-FY2024.md
  - 10k-extractor-FY2025.md
- Skills:
  - gaap-standards/ (accounting rule interpretations)
  - financial-ratios/ (calculation formulas)
  - sec-filing-structure/ (where to find specific data)
- MCP: SEC EDGAR API (fetch filings automatically)

```
> Loading JavaScript code...
```

/finance:analyze-10k TSLA 2021-2025

```
> Loading JavaScript code...
```

// Sub-agent: 10k-extractor-FY2023

Task: Extract all financial data from Tesla FY2023 10-K

Extract:

1. Income Statement (Revenue, COGS, Operating Expenses, Net Income)
2. Balance Sheet (Assets, Liabilities, Equity)
3. Cash Flow (Operating, Investing, Financing)
4. Key Metrics (mentioned in MD&A)
5. Risk Factors (new risks added this year)
6. Segment Data (Automotive, Energy, Services)

Return structured JSON with exact figures and page citations.

```
> Loading JavaScript code...
```

KEY THEMES FROM MANAGEMENT DISCUSSION:

FY2023 Margin Compression:

- Management cited "aggressive pricing strategy" to maintain market share
- Raw material costs (lithium, nickel) increased 15% YoY
- New factory ramp costs in Austin and Berlin

FY2024-2025 Recovery:

- Pricing stabilized; focus shifted to cost reduction
- Manufacturing efficiency improved (vehicles per employee up 12%)
- Energy Storage segment growing faster than Automotive (45% vs 15%)

Risk Factors Evolution:

- FY2023: Added "increased competition from legacy automakers"
- FY2024: Added "regulatory uncertainty on autonomous driving"
- FY2025: Added "supply chain concentration in China"

```
> Loading JavaScript code...
```

Plugin Name: discovery-drone

Components:

- Slash Command: /legal:discover <risk-type> <folder>
- Sub-agents (specialized by contract type):
  - contract-analyzer-msa.md (Master Service Agreements)
  - contract-analyzer-nda.md (Non-Disclosure Agreements)
  - contract-analyzer-sow.md (Statements of Work)
  - contract-analyzer-saas.md (SaaS/Subscription Agreements)
  - contract-analyzer-license.md (Software Licenses)
- Skills:
  - legal-clause-taxonomy/ (clause identification patterns)
  - risk-scoring/ (severity classification logic)
  - jurisdiction-rules/ (state/country-specific considerations)

```
> Loading JavaScript code...
```

/legal:discover indemnification-liability /contracts/vendor-agreements/

```
> Loading JavaScript code...
```

// From MSA Analyzer

```
{
    "contract": "Acme-Corp-MSA-2023.pdf",
    "clauses": [
        {
            "type": "indemnification",
            "party_indemnifying": "Us",
            "party_indemnified": "Acme Corp",
            "scope": "IP infringement, third-party claims, data breaches",
            "cap": "Unlimited",
            "page": 15,
            "exact_text": "Vendor shall indemnify, defend, and hold harmless Client
            from any and all claims...",
            "risk_score": "HIGH",
            "risk_reason": "Unlimited indemnification, one-sided (we indemnify them,
            they don't indemnify us)"
        },
        {
            "type": "liability_limitation",
            "our_cap": "12 months fees",
            "their_cap": "None specified",
            "consequential_damages": "Excluded for us, not for them",
            "page": 16,
            "risk_score": "HIGH",
            "risk_reason": "Asymmetric liability caps favor counterparty"
        }
    ]
}
```


```
> Loading JavaScript code...
```

Plugin Name: voc-engine

Components:

- Slash Command: /product:voc-cluster <ticket-export>
- Sub-agents:
  - ticket-themer.md (batch extraction via API)
  - cluster-engine.md (pattern synthesis)
  - impact-scorer.md (prioritization by revenue/churn risk)
- Skills:
  - product-taxonomy/ (feature categories, user personas)
  - sentiment-analysis/ (frustration indicators)
- MCP: Zendesk, Intercom, or Jira (live ticket access)

```
> Loading JavaScript code...
```

import anthropic

import json

client = anthropic.Anthropic()

Load tickets from export

with open('zendesk-export.json') as f:

    tickets = json.load(f)

Create batch request

```
batch = client.messages.batches.create(
    requests=[
        {
            "custom_id": f"ticket-{ticket['id']}",
            "params": {
                "model": "claude-sonnet-4-5-20250929",
                "max_tokens": 512,
                "messages": [{
                    "role": "user",
                    "content": f"""Analyze this support ticket.
                    Ticket #{ticket['id']}:
                    Subject: {ticket['subject']}
                    Body: {ticket['description']}
                    Return JSON only:
                    {{
                        "theme": "one of: mobile, web_ux, integrations, billing,
                            performance, onboarding, reporting, api, other",
                        "sentiment": "satisfied | neutral | frustrated | angry",
                        "pain_point": "1-sentence description of core issue",
                        "feature_request": "specific feature if mentioned, else null",
                        "churn_risk": "low | medium | high"
                    }}
                    """
                }]
            }
        }
        for ticket in tickets
    ]
)
print(f"Batch ID: {batch.id}")
```

print(f"Status: {batch.processing_status}")

Poll for completion, then retrieve results

```
> Loading JavaScript code...
```

/product:voc-cluster ticket-themes.json

```
> Loading JavaScript code...
```

Plugin Name: voice-dna

Components:

- Slash Command: /marketing:extract-voice <content-folder>
- Sub-agents:
  - tone-analyzer.md (emotional register, formality level)
  - vocabulary-mapper.md (word choices, phrases to use/avoid)
  - pattern-extractor.md (sentence structures, paragraph patterns)
  - example-curator.md (best examples per category)
- Skills:
  - linguistics-frameworks/ (tone dimensions, readability metrics)
  - brand-voice-templates/ (B2B, B2C, technical, casual)

```
> Loading JavaScript code...
```

/marketing:extract-voice /content/approved-2024/

```
> Loading JavaScript code...
```

Write in [Company]'s brand voice:

- Tone: Confident but approachable, expert but not arrogant
- Avoid: Jargon without explanation, superlatives (revolutionary, game-changing), passive voice
- Prefer: Short sentences (15-20 words), concrete examples, "you" over "users"
- Structure: Lead with benefit, explain how, end with CTA
- Sample: "Our research shows that teams using automated workflows save 10 hours per week. Here's how to set one up in 5 minutes."

✅ Use Case #6 — HR: The "Resume Radar"

Mission: Screen hundreds of resumes and rank candidates against job requirements

The Problem

HR teams receive 200-500 resumes per open role. Manual screening takes 30+ hours and introduces inconsistency. Good candidates get buried; bias affects decisions.

Plugin Architecture

```
> Loading JavaScript code...
```

Workflow Execution

Step 1: User Invokes Command

```
> Loading JavaScript code...
```

Step 2: Requirements Extraction

Claude parses job description:

```
> Loading JavaScript code...
```

Step 3: Parallel Resume Processing

200 sub-agents process simultaneously:

```
> Loading JavaScript code...
```

Step 4: Scoring Algorithm

```
> Loading JavaScript code...
```

Step 5: Output Generation

Excel Workbook: `PM-Candidates-Ranked.xlsx`

Tab 1: Candidate Rankings

Rank	Name	Match %	Years Exp	Required Skills	Nice-to-Have
1	Jane Smith	95%	7	6/6 ✅	Fintech, Techn
2	John Chen	92%	8	6/6 ✅	Technical, AI/M
3	Sarah Johnson	88%	6	5/6 ✅	Fintech
4	Mike Williams	82%	5	5/6 ✅	None
5	Emily Brown	78%	4	5/6 ✅	Technical
...	...	...	...	...	...
45	Alex Turner	45%	2	2/6	None

Tab 2: Skills Gap Analysis

Skill	Candidates With	Candidates Without	Gap Severity
SQL	156 (78%)	44 (22%)	Low
Agile/Scrum	134 (67%)	66 (33%)	Medium
B2B SaaS	112 (56%)	88 (44%)	High
Stakeholder Mgmt	178 (89%)	22 (11%)	Low

Tab 3: Diversity Metrics (Anonymized)

Category	Pool Composition	Advancing (Top 20)	Notes